

Programming C — (Modules 1–5 + Linux Shell Basics)

This document contains the **first block of lectures** for an introductory C programming course.

Covered modules:

1. Computers and the Von Neumann Architecture
2. Representation of Information
3. Algorithms and Programming
4. Introduction to C
5. Structure of a C Program and Compilation
6. The Linux Shell and its Base Commands

These lectures introduce the **foundations necessary to understand programming**, before students begin writing more complex C programs.

MODULE 1 — Computers, Programs, and the Von Neumann Model

What is a Computer?

A **computer** is a programmable machine capable of:

- receiving **input**
- **processing** information
- **storing** data
- producing **output**

PROF A computer executes a sequence of instructions called a **program**.

Examples of computers:

- laptops
- smartphones
- servers
- embedded systems (cars, washing machines, routers)

Although these machines look different, internally they share a **similar logical structure**.

Programs

A **program** is a sequence of instructions that tells the computer what to do.

Example tasks performed by programs:

- calculating bank transactions
- controlling robots
- displaying web pages
- running video games

Programs are written using **programming languages**, such as:

- C
- Python
- Java
- C++

These languages must ultimately be translated into **machine language** that the processor understands.

The Von Neumann Architecture

Most modern computers follow the **Von Neumann architecture**, proposed by **John von Neumann (1945)**.

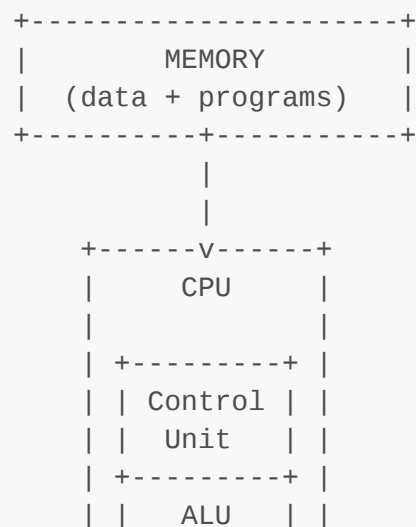
The key idea is the **stored-program concept**:

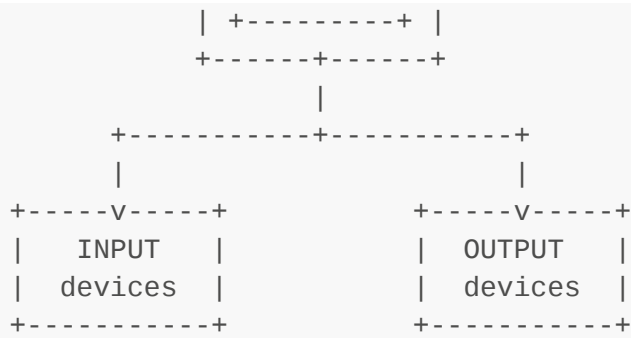
- both **instructions** and **data** are stored in memory

This architecture contains four main components:

1. **CPU (Central Processing Unit)**
 2. **Memory**
 3. **Input devices**
 4. **Output devices**
-

Diagram of a Von Neumann Machine





The CPU executes instructions stored in memory and communicates with input/output devices.

CPU Components

Control Unit

Responsible for:

- fetching instructions
- decoding instructions
- coordinating operations

Arithmetic Logic Unit (ALU)

Performs:

- arithmetic operations (addition, subtraction)
- logical operations (AND, OR, NOT)

The Fetch–Decode–Execute Cycle

Program execution follows a repetitive cycle:

1. **Fetch** instruction from memory
2. **Decode** instruction
3. **Execute** instruction
4. Store results

Example instruction:

```
ADD R1, R2
```

Meaning:

Add values in registers R1 and R2.

This cycle repeats **billions of times per second**.

Machine Language

The processor understands only **machine language**.

Machine language instructions are sequences of **binary numbers**.

Example machine instruction (x86 architecture):

```
10110000 01100001
```

This instruction actually consists of **two bytes**.

- 10110000 → opcode
- 01100001 → data value

The corresponding assembly instruction is:

```
MOV AL, 61h
```

Meaning:

Move the value **0x61** into the register **AL**.

Why is 97 written as 61h?

The suffix **h** indicates a **hexadecimal number**.

Hexadecimal is base-16 and is widely used in computer science because it is compact and easy to convert to binary.

Digits in hexadecimal:

```
0 1 2 3 4 5 6 7 8 9 A B C D E F
```

Each hexadecimal digit corresponds to **4 bits**.

Converting 97 to Hexadecimal

First convert 97 to binary:

```
97 = 01100001
```

Now group bits into blocks of 4:

```
0110 0001
```

Convert each group to hexadecimal:

```
0110 = 6  
0001 = 1
```

Therefore:

```
97 (decimal) = 61 (hexadecimal)
```

Assembly notation writes this as:

```
61h
```

So the instruction loads the value **97** into the register.

MODULE 2 — Representation of Information

Why Binary?

Computers represent information using **binary digits (bits)**.

PROF

A bit can have two possible values:

```
0  
1
```

Reasons for using binary:

- electronic circuits easily represent two states
- more reliable in the presence of noise
- simpler hardware implementation

Physical representation:

- voltage high

- voltage low

Bits and Bytes

Basic information units:

Bit = binary digit

Byte = **8 bits**

Example byte:

```
01000001
```

Interpretations:

Decimal number:

```
65
```

ASCII character:

```
'A'
```

Binary Numbers

Binary numbers are **base-2** numbers.

Example:

```
1011
```

Calculation:

```
1×23 + 0×22 + 1×21 + 1×20
```

Result:

```
8 + 0 + 2 + 1 = 11
```

Integer Representation

Integers are stored using a **fixed number of bits**.

Example: **8-bit unsigned integer**

Range:

0 → 255

Example values:

```
00000000 = 0
00000001 = 1
00000101 = 5
11111111 = 255
```

Signed integers typically use **two's complement representation**.

Two's Complement Representation

Two's complement is used to represent **negative integers**.

Procedure to obtain $-x$:

1. Write the binary representation of x
2. Invert all bits
3. Add 1

PROF

Example — Represent -5

Step 1: write +5

00000101

Step 2: invert bits

11111010

Step 3: add 1

```
11111011
```

So:

```
11111011 = -5
```

Example — Addition with Two's Complement

One major advantage of two's complement is that **the same hardware can perform both addition and subtraction.**

Example computation:

```
5 + (-5)
```

Binary representation (8 bits):

```
  00000101   (5)
+ 11111011  (-5)
-----
1 00000000
```

The leftmost carry bit is **discarded** in fixed-size arithmetic.

PROF

Result stored in 8 bits:

```
00000000
```

Which correctly represents:

```
0
```

Floating Point Numbers

Real numbers are represented using **floating-point numbers.**

IEEE 754

$$6.25 = 1.5625 \times 2^2$$

Sign	Exponent	Mantissa
1bit	8bits	23bits

| S | EEEEEEEE | MMMMMMMMMMMMMMMMMMMMMM |

$$(-1)^S \times 1.M \times 2^{(E - \text{bias})}$$

PROF

6.25 (decimal) = 110.01 (binary)

$$110.01 = 1.1001 \times 2^2$$

Step 3: Determine fields.

Sign bit:

$S = 0$ (number is positive)

Exponent:

$E = 2 + 127 = 129$

129 in binary:

10000001

Mantissa (fraction after leading 1):

100100000000000000000000

Final IEEE 754 representation:

0 10000001 100100000000000000000000

In hexadecimal this becomes approximately:

40C80000

PROF

MODULE 3 — Algorithms and Programming

What is an Algorithm?

An **algorithm** is a finite sequence of instructions used to solve a problem.

Properties:

- finite
- deterministic
- unambiguous

Example: compute maximum of two numbers.

Algorithm:

1. read A
2. read B
3. if A > B print A
4. else print B

Algorithms vs Programs

Algorithm:

- abstract description of a solution

Program:

- implementation of the algorithm in a programming language

Example language:

C

MODULE 4 — Introduction to C

The C Programming Language

C is a **general-purpose programming language**.

Developed in the **1970s**.

Key characteristics:

- compiled language
- efficient
- portable

Widely used in:

- operating systems
- embedded systems
- compilers

History of C

Created by **Dennis Ritchie** at **Bell Laboratories**.

Original goal:

implement the **UNIX operating system**.

Influenced many languages:

- C++
- Java
- C#

MODULE 5 — Structure of a C Program

First C Program

```
#include <stdio.h>

int main() {

    printf("Hello world!\n");

    return 0;

}
```

This program prints text on the screen.

Program Structure

Important components:

`#include <stdio.h>`

Includes the **standard I/O library**.

`main()`

Entry point of the program.

`printf()`

Function used to display output.

Compilation Overview

C is a **compiled language**.

The program must be translated into **machine code** before execution.

Compilation involves several steps.

Step 1 — Preprocessing

Handles directives beginning with `#`.

Examples:

```
#include  
#define
```

Operations:

- include header files
- replace macros
- remove comments

Step 2 — Compilation

The compiler:

- checks syntax
- checks types
- generates **assembly code**

Step 3 — Assembly

Assembler converts assembly into **machine code**.

Output file:

```
program.o
```

(object file)

Step 4 — Linking

The linker combines:

- object files
- external libraries

Example:

`printf()` is provided by the **C standard library**.

GCC Example

Compile a program:

```
gcc hello.c -o hello
```

Run the program:

```
./hello
```

The output executable contains **machine instructions executed by the CPU**.

MODULE 6 — Basic Linux Shell Commands

Operating System Concept

An **OS** manages resources and provides services:

- CPU, memory, disks
- files/directories
- input/output devices
- program scheduling

Examples: Linux, Windows, macOS

The **shell** is a program that emulates a **CLI** (typical of UNIX-like systems)

Linux File System Layout (Vertical)

```
/
├── bin
├── boot
├── dev
├── etc
├── home
│   ├── alice
│   └── student
│       ├── documents
│       ├── programs
│       └── hello.c
```

```
|      └─ downloads
|      └─ lib
|      └─ tmp
|      └─ usr
```

This vertical view shows the hierarchy from root `/` down to nested directories and files.

Current Directory & Paths

```
pwd # print working directory
/home/student
```

Absolute path: `/home/student/programs/hello.c`

Relative path: `programs/hello.c`

Special symbols:

```
.    current directory
..   parent directory
~    home directory
```

Examples:

```
cd ..          # move up one level
cd ../..       # move up two levels
cd student/programs # move down relative
```

PROF

Basic Commands with Examples

- `ls` → list files
 - Example: `ls` → shows all files in current directory
 - Example: `ls -l` → detailed listing with permissions, owner, size, date
 - Example: `ls -a` → includes hidden files
- `mkdir` → create directory
 - Example: `mkdir projects` → creates a new directory named projects
- `touch` → create empty file
 - Example: `touch notes.txt` → creates an empty file notes.txt

- `cp` → copy file
 - Example: `cp notes.txt backup_notes.txt` → copies notes.txt to backup_notes.txt
- `mv` → move or rename file
 - Example rename: `mv notes.txt old_notes.txt`
 - Example move: `mv old_notes.txt projects/`
- `rm` → remove file or directory
 - Example file: `rm notes.txt`
 - Example directory: `rm -r old_project/`
- `cat` → view file content
 - Example: `cat hello.c` → prints the contents of hello.c to the terminal
- `less` → scrollable view
 - Example: `less hello.c` → allows scrolling through hello.c interactively

Typical Workflow

```
mkdir c_course
cd c_course
vi hello.c
gcc hello.c -o hello
./hello
```

Steps:

1. create directory
2. write program
3. compile
4. execute

PROF

End of Part 1 (Modules 1–6)

Next lectures:

- variables, data types, operators
- control flow, functions
- arrays, pointers
- structures

..etc

